PROJECT CONTEXT DOCUMENT

ReLoop

No donation reaches a dead end.

A location-aware donation platform that ranks recipients on multiple criteria, keeps re-routing an item after every rejection, and falls back to monetised recycling — so every donation ends in reuse or material recovery, and the platform can say which.

OWNER

Fareeha Amin

DOMAIN

Circular economy · reusable household goods

CONTEXT

Final-year BE project · Dept. of CSE,
City Engineering College, Bengaluru

TARGET STACK

FastAPI · PostgreSQL · React ·
Scikit-learn / TensorFlow

DOCUMENT

v2.0 · October 2026

PURPOSE

Reference context for any new conversation, build session or contributor

Contents

01 Project at a glance
02 The problem
03 The solution
04 Scope
05 Design principles
06 Actors
07 Donation lifecycle
08 Matching engine
09 Recycling fallback
10 Impact accounting
11 Logistics & money
12 Data model
13 Tech stack
14 Decided against (and why)
15 Security & privacy
16 Build plan
17 Future modules
18 Risks & mitigations
19 Lessons from the earlier prototype
20 Academic context
21 Glossary
22 Sources

How to use this document

Background for anyone — human or AI — joining the ReLoop build. It explains the problem, the domain model, the mechanisms and the decisions already taken, so a fresh conversation starts informed instead of from zero.

It is **not** a build instruction and not a frozen specification. The build prompt is a separate document. Where this document and a better idea disagree, the better idea wins — say so and explain why.

01 Project at a glance

Donation platforms are good at listing items and bad at what happens next.

Someone has a bag of children's clothes. They post it. An NGO is notified. The NGO already has more clothes than storage and declines — or simply never replies. At that point most platforms have nothing further to say. The listing goes stale, the donor loses interest, and the clothes go in the bin. The platform recorded an intention and produced no outcome.

ReLoop treats a rejection as a defined transition rather than an ending. When a recipient declines, the item is re-ranked against the remaining candidates with that recipient excluded. If candidates keep declining, the donor is brought back in with a decision to make. If reuse genuinely fails, the item is routed to a recycling partner and the donor is credited for the recovered material value.

WHAT IT DOES

- Geocodes donors and recipient organisations
- Ranks eligible recipients on six factors, with reasons
- Re-routes after every rejection
- Escalates to the donor at a threshold
- Falls back to paid recycling
- Accounts for mass diverted and carbon offset

WHAT MAKES IT DIFFERENT

- Rejection is a transition, not an end
- Declined items gain visibility, not lose it
- Two terminal states, no limbo
- The score explains itself
- Reward even when reuse fails
- Unroutable items surface, never vanish

INTENDED OUTCOME

- Fewer donations abandoned mid-flow
- Better recipient fit than nearest-first
- Hard-to-place categories still move
- A record of what happened to every item
- Impact figures with a published method
- A loop that measurably closes

02 The problem

70.73 lakh

tonnes of textile waste generated in India a year. **58% is post-consumer** — the clothes households actually finish with, which is exactly ReLoop's domain.

Ministry of Textiles, *Mapping of Textile Waste Value Chain in India*, 2026

~45%

of post-consumer textile waste is **not** diverted from landfill. Only "nearly 55%" is, against 95% recovery for factory-floor waste — the consumer end is where the system leaks.

Ministry of Textiles, 2026

22%

of India's municipal solid waste is **unaccounted** — 37,373 of 1,70,339 tonnes a day neither processed nor landfilled. Things leave the record entirely.

CPCB, MSW Annual Report 2021–22

Where the journey breaks

Pain point	What happens
Weak matching	Recipients are ranked by distance alone, or not ranked at all. An orphanage two streets away with no storage outranks one four kilometres away that urgently needs exactly this.
Rejection is a dead end	The recipient declines and the platform has no next step. The item sits listed until the donor gives up.
Hard-to-place items sink	Whatever is awkward — adult clothing, worn toys, textbooks for a superseded syllabus — gets declined once and then ranks lower everywhere, forever.
No outcome record	Nobody can say whether a donation was reused, recycled or binned. Impact claims rest on listings created, not outcomes achieved.
Reuse sometimes genuinely fails	No recipient wants it, and there is no path from "unwanted" to "materially recovered".

Solved by

Pain point	Mechanism
Weak matching	Hard filters, then a six-factor score — proximity, declared need, urgency, capacity, acceptance history, rating
Rejection is a dead end	Exclusion list plus re-rank against the remaining candidates
Hard-to-place items sink	**Rejection boost** — the score rises with each decline, capped
No outcome record	Explicit lifecycle with two terminal states, plus an append-only audit log of transitions
Reuse sometimes genuinely fails	Recycling fallback with settlement — donor credit and platform fee from recovered material value
Items vanish anyway	Unroutable donations surface in an admin queue as a first-class feature

03 The solution

One continuous loop. The highlighted step is where ReLoop departs from a listings board.

1 List	2 Rank	3 Offer	**4 Re-rank**	5 Escalate	6 Recover	7 Account
category, condition, photos, pickup window	filter, then score with reasons	notify the top candidate	on decline: exclude, boost, offer again	donor decides after N declines	recycle and settle if reuse fails	mass, carbon, completion

> **Positioning line**
>
> ReLoop doesn't stop at listing an item. It ranks recipients on what they actually need, keeps re-routing after each rejection, brings the donor back in when candidates run out, and pays for the material when reuse fails — so nothing is left in limbo.

04 Scope

IN SCOPE — MVP

- Categories: **clothes, toys, books**
- Donor, Acceptor, Admin roles
- Geocoded addresses, map discovery
- Ranked matching with rejection handling
- Full lifecycle with live status
- Recycling fallback with donor credit
- Three logistics paths, one simulated
- Simulated payments and commission
- Impact estimates and gamification
- Admin verification and unroutable queue

OUT OF SCOPE — DELIBERATELY

- **Food donation.** Cold chain, expiry and food-safety handling the platform does not model
- **Live logistics APIs.** Simulated behind a provider abstraction
- **Real payment gateway.** Simulated and labelled as such
- **Trained ML models.** No accept/reject history exists at launch
- **Native mobile apps.** Responsive web first
- **Resale / thrift.** Planned as a module — see §17

05 Design principles

Non-negotiable. If a change would break one of these, it needs a conversation, not a commit.

1. **Every donation reaches a defined end state.** Exactly two terminal states — reuse or material recovery. No item can enter a state it cannot leave.
2. **The score explains itself.** Ranking is decomposable into its factors and surfaced as reasons, never as a bare number.
3. **Rejections are data.** Never delete or overwrite a `Match` row.
4. **One source of truth for every constant.** Category weights, carbon factor, fee percentages, thresholds — defined once, imported everywhere.
5. **The lifecycle is an explicit transition table**, not conditionals scattered across route handlers.
6. **A stub says it is a stub** — in the code, in the comments and in the README. No `TODO` presented as a shipped feature.
7. **Nothing fails silently.** An unroutable donation surfaces to an admin.
8. **Estimates are labelled as estimates.** Mass and carbon figures are modelled, not measured.
9. **Config over hardcoding.** Weights, caps, thresholds and fees live in configuration so they can be tuned and measured.
10. **Donor addresses are sensitive by default.** Precise location is revealed on need, not on listing.
11. **Scoring and lifecycle logic are pure functions**, decoupled from the database, so they are unit-testable and reusable in a simulation.
12. **The API is the only source of truth.** No business logic in the React layer — a future mobile app must be able to use the same endpoints.

06 Actors

Donor

An individual with surplus goods. Registers, is geocoded at signup, creates a donation (category, quantity, condition, photos, pickup address, preferred time window, optional priority flag and deadline), then receives a **ranked list of suitable acceptors** and either takes the top suggestion or picks another from a map. Tracks the item through its full lifecycle. Accrues impact points and wallet credit.

Acceptor — NGO, orphanage, school, shelter, institution

Registers with organisation name, registration ID, verification documents, contact person, geocoded address, storage capacity, pickup capability and categories accepted. Maintains a **current needs list** — category, quantity needed, urgency, active flag — which is the main input to matching, so it must be trivial to keep current. Works a single queue of incoming offers: accept or decline. Schedules its own pickups where it can collect. Only **admin-verified** organisations enter matching at all.

Admin

Verifies organisations, approves institutional accounts, monitors commission and platform-wide impact, flags abuse, suspends users — and watches the **queue of donations that could not be routed anywhere**. That queue is a feature, not an error log: it is where the closed loop meets reality.

07 Donation lifecycle

The heart of the system.

```
LISTED → MATCHED → ACCEPTED → PICKUP_SCHEDULED → IN_TRANSIT → DELIVERED → COMPLETED
            ↓
       RE_MATCHING ⇄ MATCHED
            ↓
        ESCALATED
            ↓
    WASTE_REDIRECTED → RECYCLED

plus CANCELLED
```

State	Meaning
`LISTED`	Created; candidates not yet ranked.
`MATCHED`	Top-ranked acceptor notified, awaiting a response.
`RE_MATCHING`	Declined. The rejector joins this donation's exclusion list and is never offered it again. Re-rank against the remainder.
`ESCALATED`	Rejection threshold reached (default 3). The donor decides.
`ACCEPTED`	Acceptor confirmed; logistics to arrange.
`PICKUP_SCHEDULED` → `IN_TRANSIT` → `DELIVERED`	Collection and handover.
`COMPLETED`	**Terminal — reuse.** Impact recorded.
`WASTE_REDIRECTED`	Routed to a recycling partner; settlement computed.
`RECYCLED`	**Terminal — material recovery.**
`CANCELLED`	Donor withdrew.

Rejection handling, step by step

1. Record the rejection as a `Match` row with `response = REJECTED`. Never delete it.
2. Add that organisation to the donation's exclusion list.
3. Increment `rejection_count`; apply the rejection boost.
4. Re-rank the remaining eligible candidates; offer to the new top choice.
5. At the threshold, escalate to the donor with three options: **re-match** · **upgrade to priority** · **redirect to recycling**.
6. A scheduled sweeper handles donations that time out past their deadline.

> **The test that matters**
>
> Assert that every non-terminal state has at least one outgoing transition, and that `COMPLETED` and `RECYCLED` have none. A donation that can enter a state it cannot leave is the single bug that breaks the entire premise. Write that test before the UI exists.

08 Matching engine

When a donation is created, score every *eligible* acceptor and return a ranked list with reasons.

Hard filters, applied before scoring

Cheap eliminations first: **admin-verified**, **accepts this category**, **has remaining capacity**, and **not on this donation's exclusion list**.

Scoring factors

Factor	Direction
Geographic proximity (haversine or PostGIS)	Nearer scores higher
Match against the acceptor's current needs list	Declared need scores higher
Urgency of that need	More urgent scores higher
Remaining storage capacity	More headroom scores higher
Historical acceptance rate	Reliable responders score higher
Donor rating of the organisation	Better rated scores higher

Two multipliers

Priority boost — a donor-flagged urgent donation ranks higher.

Rejection boost — the score rises with each decline, capped so it cannot dominate the ranking. Magnitude and cap configurable, with a flag to turn it off so its effect can be measured rather than assumed.

On the weights and on the ML

- Start with **assigned** weights. A new platform has no accept/reject history; there is nothing to learn from. Say so in the code.
- Once history accumulates, calibrate from observed outcomes — that is what the Scikit-learn layer is for.
- Demand forecasting sits behind a clean interface with a deterministic neutral fallback, so the platform runs correctly before any model exists.
- Keep the score **decomposable**. "Ranked first because: 2.1 km away, clothes listed as a HIGH-urgency need, 80% storage free" is worth more to a donor than a number.

> **Open to better ideas**
>
> A weighted sum is the obvious starting point, not the only option. Learning-to-rank, bandit allocation, or a fairness constraint that stops one large NGO winning every match are all legitimate alternatives. Propose, explain the trade-off, build the better one.

09 Recycling fallback

Triggered when all ranked candidates decline, the candidate list is empty, the donor chooses redirection at escalation, or the deadline passes.

1. Select the nearest **active** waste-management partner that accepts the category.
2. Estimate mass: `quantity × per-category weight`.
3. Compute recovered value from the partner's rate; split into a **donor credit** and a **platform fee**.
4. Credit the donor's wallet; move the donation to `RECYCLED`.

The donor is rewarded for donating even when reuse fails. That is what brings them back after a bad experience, and it is why the recycling path is a feature rather than a consolation prize.

> **Do not pretend**
>
> If no eligible recycler is reachable, the donation must surface in the admin queue. The guarantee ReLoop offers is a property of the *workflow*, not of physical reality — there is not always a partner. The code should be honest about that rather than silently marking something recycled.

10 Impact accounting

```
waste_diverted_kg = quantity × CATEGORY_WEIGHT[category]
carbon_offset_kg  = waste_diverted_kg × CARBON_FACTOR
completion_rate   = (completed / total) × 100
```

Dashboards show items donated, waste diverted, carbon saved, beneficiaries reached, completion rate, and an impact badge (Bronze → Silver → Gold → Platinum). Leaderboards for top donors, top organisations, top cities.

> **Single source of truth**
>
> Define `CATEGORY_WEIGHT` and `CARBON_FACTOR` in exactly **one** module and import it everywhere. Store impact **per donation**, not only as an aggregate — a future carbon-credit or CSR module needs per-item records, and they cannot be reconstructed later.

These figures are **estimates**, not measured emissions. Label them that way in the UI, and publish the constants and the method alongside them.

11 Logistics & money

Three pickup paths, donor chooses

1. **Platform-arranged pickup** — simulate a third-party booking. Compute distance and estimated cost, create a booking record, take simulated payment, advance through Scheduled → Picked Up → Delivered. An 8–12% platform fee is recorded in a `Commission` table. A `LogisticsProvider` model (name, endpoint, commission %, status) exists so a real API can slot in later without a rewrite.
2. **Acceptor self-scheduled pickup** — the acceptor accepts, picks a slot and drives the status itself.
3. **Donor drop-off** — marked self-delivered.

An optional **pickup scheduler** suggests a slot from the donor's preferred window, acceptor availability, and mock travel-time estimation.

Payments

A `Payment` model (user, donation, amount, logistics cost, platform commission, status, simulated transaction ID) with a Razorpay-style mock checkout that auto-succeeds. Labelled as a simulation in the UI *and* in the API response, so no downstream consumer mistakes it for a settled transaction.

12 Data model

Entity	Notes
`User`	Custom model with a `role` field — Donor / Acceptor / Admin. Not a single `is_staff` flag.
`OrganizationProfile`	Verification status, storage capacity, categories accepted, pickup capability.
`CurrentNeed`	Per-organisation needs list: category, quantity, urgency, active. Drives matching.
`Donation`	Plus status, exclusion list, `rejection_count`, priority flag, deadline.
`DonationImage`	Object storage in production, not database blobs.
`Match`	**One row per candidate offered** — score, rank, response, timestamp.
`PickupRequest`, `LogisticsProvider`	Collection and provider abstraction.
`Payment`, `Commission`, `Wallet`, `WalletTxn`	Simulated money. Generalise cleanly — a resale module reuses all four.
`ImpactMetrics`	Per donation, computed from the single-source constants.
`WasteCollector`	Recycling partners: rates, accepted categories, active status, location. Rates live on the record, not in code.
`DriveEvent`	Corporate and school donation drives.
`Review`	Ratings, which feed back into scoring.
`StatusEvent`	Append-only audit of every lifecycle transition: actor, from, to, timestamp, reason.

> **Never delete a Match row**
>
> Including — especially — rejections. The accept/reject history is what trains the acceptance model, calibrates the ranking weights and feeds demand forecasting. It is the most valuable data the platform will ever produce, and it exists only if nothing overwrites it.

13 Tech stack

Layer	Choice	Why
Backend	FastAPI, Python 3.11+	Async where it helps, typed request/response models, OpenAPI for free

ORM & migrations	SQLAlchemy 2.x + Alembic	Explicit schema history; migrations reviewable in diffs
Validation	Pydantic v2	One schema definition shared by API docs, validation and tests
Database	PostgreSQL 15+, PostGIS optional	Relational integrity for the lifecycle; PostGIS if geo queries become the bottleneck
Auth	JWT, access + refresh	Stateless, works unchanged for a future mobile client
Background work	Celery + Redis *(from Phase 2)*	Deadline sweeps, notifications, re-ranking. Deferred until there is something to schedule
Frontend	React 18 + Vite, Tailwind	One codebase, fast dev loop, installable as a PWA later
Data fetching	TanStack Query	Cache and refetch for live status without hand-rolled polling
Maps	Google Maps or Mapbox	Geocoding plus the discovery map; cache every geocode result
Intelligence	Scikit-learn, TensorFlow	Weight calibration and demand forecasting — behind interfaces with deterministic fallbacks
Notifications	Firebase Cloud Messaging	Web push with no app store; email as fallback
Testing	pytest	Scoring and lifecycle as pure, DB-free functions
Config	`.env`	No secrets in code; nothing assumes a container or a host

14 Decided against (and why)

Not doing	Reason
Django + MySQL	A Django/MySQL prototype exists from an earlier workshop. It is not being extended and is not a reference implementation. This is a rebuild.
Docker Compose at the start	Postgres and Redis run fine natively or on a managed free tier. Add a compose file when a second machine or a deploy makes it worth it — and never let code assume a container.
Celery and Redis in Phase 1	The only early scheduled job is the deadline sweep. A plain function with an admin trigger is testable and has fewer moving parts.
Learned ranking weights at launch	No history exists to learn from. Shipping a "trained" model with no training data is the dishonesty this project is trying to avoid.
A real payment gateway	Live settlement adds compliance and integration burden without changing what the platform demonstrates. The abstraction is real; the backend is mocked and labelled.
Live courier APIs	Same reasoning. `LogisticsProvider` exists so a real integration drops in without a rewrite.

Blockchain traceability	An append-only `StatusEvent` table gives the same auditability at a fraction of the cost. Revisit only if an external party must verify records without trusting the platform.
A vector database / RAG	Nothing in the matching path is a semantic search problem. Structured fields and rules are both faster and explainable.
Native mobile apps	Responsive web first, PWA second. Keeping all logic behind the API means a native client later costs no backend rework.
Pruning old `Match` rows	Tempting for table size, fatal for the product. That history is the asset.

15 Security & privacy

Area	Design
Access	JWT with refresh; role-based route dependencies enforced per role, not via a single admin flag.
Donor addresses	Treated as sensitive. Approximate location during matching; precise address released to the acceptor only after acceptance, and logged when it is.
Uploads	Image type and size validated server-side; stored in object storage with non-guessable keys, never served from a predictable path.
Audit	Append-only `StatusEvent` row for every consequential action: actor, action, donation, result.
Abuse	Rate limiting on donation creation and on accept/reject, so neither queue can be flooded.
Verification documents	Organisation paperwork is reviewed by an admin and then not needed again — store metadata, minimise retention.
Consent & retention	Explicit consent before storing an address; a stated retention period; deletion that actually deletes. DPDP Act principles: consent, purpose limitation, deletion.

16 Build plan

Six phases, built and reviewed one at a time.

Phase	Contents
0 — Skeleton	Folder structure, project docs, `.gitignore`, and a reachable database. Roughly fifteen minutes. No application code.
1 — Foundation	Scaffold, models, migrations, JWT auth with three roles, donation CRUD, organisation registration and needs list, pytest.
2 — Core loop	Matching engine, lifecycle state machine with the transition test, rejection handling and re-matching, recycling fallback, deadline sweeper.
3 — Experience	React frontend, map discovery, dashboards, status tracking, notifications.

4 — Logistics & money	Three pickup paths, simulated payments, commission, wallet.
5 — Scale & intelligence	Impact analytics, leaderboards, corporate drives, weight calibration from history, demand forecasting, admin tooling.

Phase 0 in detail

Deliberately thin. Phase 0 exists so the repo has a shape agreed before code lands on top of it — not to provision the whole platform.

Item	Detail
Structure	`backend/`, `frontend/`, `docs/`, and later `sim/`. Settled now so nothing has to move.
Repo hygiene	GitHub repo, protected `main`. `.gitignore` covering `.env`, `__pycache__`, `node_modules`, `*.sqlite3`, build output. Checked before the first push, not after.
Project docs	`CLAUDE.md` (decisions, commands, env vars), `PROMPT.md` (the build prompt), `docs/` (this document). README with pinned Python and Node versions.
Database	PostgreSQL 15+ — local install or a managed free tier (Neon, Supabase). Reachable from Python with one `SELECT 1`.
Environment	`.env` and `.env.example` with only what exists today: `DATABASE_URL`, `SECRET_KEY`.
Tests	`pytest` configured and green on one trivial test.

> **Phase 0 is done when**
>
> The repo has a structure worth keeping, the docs are in place, and the database answers a query from application code. That is all.

Everything else arrives when it is needed

Maps keys, Firebase, Redis and the frontend toolchain are **not** Phase 0 items. Provisioning a service weeks before anything calls it buys nothing and produces a credential nobody has tested in anger.

Service	Acquired in
Maps / geocoding key *(+ spend alert)*	The phase that first geocodes an address
Redis	Phase 2, with Celery
Frontend toolchain, dev server	Phase 3
Firebase / FCM credentials	Phase 3, with notifications
Object storage for images	When images stop being a local folder

> **Two rules instead of a setup checklist**
>
> **A credential is created in the phase that first calls the service** — and the first thing built against it is the call that proves it works.
>
> `.env.example` **gains a variable the moment code reads it, never before.** A speculative environment file goes stale within a week and then lies about what the project needs, which is the same failure as a stub advertised as a feature.

Working rhythm

One phase at a time. A short plan first, then the build, then tests, then the exact run commands and a hand-verifiable acceptance check — then stop for review. Decisions, commands and environment variables land in `CLAUDE.md` as they are made, not afterwards.

Optional, high value: a simulation harness

A `sim/` package that runs synthetic donations and organisations through the **real** scoring and lifecycle code — not a reimplementation — and reports: fraction redistributed vs. recycled vs. abandoned, mean offers before acceptance, mean distance travelled, load fairness across organisations, completion rate, response time under concurrency. Compare random assignment, nearest-only, score without re-matching, full ReLoop, and full ReLoop with the rejection boost.

It is the fastest way to find out whether the rejection boost actually helps, and it turns design claims into measured ones.

17 Future modules

ReLoop is a core loop plus modules. None of the following should require touching the lifecycle engine or the scoring module — if one would, that is a design smell worth fixing now rather than later.

Module	What it adds	Hooks to leave in place
Thrift store *resale of donated goods*	Items that nobody needs for free, but that have resale value, are listed for sale instead of recycled. Proceeds fund the platform, the donor, or a chosen cause.	A **third terminal state** (`SOLD`) — so the lifecycle must accept new terminal states without rework. Listing price, condition grade and inventory fields on `Donation`. `Payment` / `Commission` / `Wallet` generalised beyond logistics fees. A policy decision on who receives proceeds.
Instant pickup *on-demand collection*	A donor requests collection now rather than scheduling a window; the nearest available courier or volunteer is dispatched live.	Fast geo-queries — this is the strongest argument for **PostGIS**. A courier availability and location model. Status pushed over websockets rather than polled. Dispatch kept separate from matching: they are different problems that look alike.

Volunteer network	People rather than couriers handle collection, which cuts cost and builds community.	Volunteer profiles with availability windows, ratings and background-check status. Reuse `LogisticsProvider` as the abstraction rather than adding a parallel path.
Food donation	Cooked and packaged surplus, with a far tighter clock.	Category rules in **configuration, not branches**: expiry window, max transit time, cold-chain flag. Deadline handling already exists — food simply has a much shorter one.
Carbon-credit / CSR reporting	Verifiable impact records suitable for an external auditor or a corporate report.	Impact stored **per donation**, with the constants and method versioned. Aggregates can always be recomputed; per-item records cannot be reconstructed.
Condition verification	Image-based grading of donated items, reducing disputes and bad matches.	Slot it behind the same interface pattern as the ranking model, with a deterministic fallback. Never block a donation on a model call.
Multilingual	Kannada and Hindi alongside English.	Every user-facing string externalised from day one. Retrofitting this is tedious; preventing it costs nothing.
Native apps	Android and iOS clients.	No business logic in the React layer. The API stays the only source of truth.

> **The test for any new module**
>
> Could it be added without editing the transition table or the scoring function? If yes, the architecture is holding. If no, find out why before the module is built — not during.

18 Risks & mitigations

Risk	Mitigation
Geocoding quota or cost runs away	Cache every geocode by normalised address; one lookup per address, ever. Spend alert from Phase 0.
No verified acceptors in a region at launch	Seed verified partners before a city goes live; the admin queue catches whatever still cannot route.
Rejection boost starves genuinely good matches	Capped, config-flagged, and measured against the plain score in the simulation harness before it is trusted.
One large NGO wins every match	Load fairness as an explicit metric; a fairness constraint or allocation cap in scoring if the data shows it.
Acceptors let their needs lists go stale	Staleness decay on need records, plus a nudge when a need has gone untouched. A stale list quietly poisons every ranking.

Donor address exposure	Precise location released only after acceptance, and logged when released.
Recycler rates drift from reality	Rates stored on the partner record and versioned, never hardcoded.
Simulated payments mistaken for real ones	Labelled in the UI and in the API response, not just in a code comment.
No trained models at launch	Deterministic fallbacks everywhere; nothing in the critical path depends on an ML call succeeding.
Impact figures challenged at review or viva	Publish the constants, the method and the source. "Estimated using 0.5 kg per clothing item" survives scrutiny; an unexplained number does not.
Scope creep from the module list in §17	Phases are reviewed one at a time. A module is a conversation, not a commit.

19 Lessons from the earlier prototype

Real bugs from the Django build, worth not repeating. Each one is the reason a principle in §05 exists.

What went wrong	What it cost
Duplicated constants. Books weighed 0.8 kg in the recycling path and 0.4 kg in the impact calculator.	The same donation produced different impact numbers depending on which code path ran. Nobody noticed until the two were compared.
A stub advertised as a feature. `validate_with_ai()` returned "not implemented in MVP" while the README promised AI image validation.	Anyone reading the README formed a false picture of what existed — including, eventually, the people who wrote it.
Undocumented magic. The rejection boost existed, buried in a scoring function, with no comment and no way to disable it.	The best idea in the codebase was invisible, untested and unmeasurable.
Lifecycle by conditionals. Status transitions scattered across view functions.	No single place to see which states connected to which, so the central claim of the project could not be verified by reading the code.

20 Academic context

A short paper describing an early design of ReLoop has been submitted to IRJMETS for review, co-authored with the project guide and four classmates. It exists to satisfy a final-year departmental requirement.

> **Important for anyone building this**
>
> The paper is **not a specification and not a constraint**. It is a snapshot of an early design, written before implementation, and it reports no measured results. The build is free to diverge from it, exceed it, or restructure it entirely. Do not treat anything in it as a requirement, and do not design backwards from it.

If the build produces something materially better than what the paper describes — which it should — that is a result worth writing up separately, not a problem to reconcile.

21 Glossary

Term	Meaning
Acceptor	Any recipient organisation — NGO, orphanage, school, shelter.
Needs list	An acceptor's declared current requirements. The main matching input.
Exclusion list	Organisations that have declined a specific donation and will never be offered it again.
Rejection boost	The multiplier that raises a donation's ranking score with each decline.
Escalation	Returning the decision to the donor once the rejection threshold is reached.
Fallback	The recycling path taken when reuse fails.
Settlement	Splitting recovered material value into donor credit and platform fee.
Closed loop	The property that every donation ends in reuse or material recovery, never in limbo.
Terminal state	A lifecycle state with no outgoing transition. There are exactly two.
Hard filter	An eligibility check applied before scoring, so ineligible candidates are never ranked.
Haversine	Great-circle distance between two lat/lng points on a sphere.
PostGIS	PostgreSQL extension for spatial types and indexed geographic queries.
PWA	Progressive Web App: a site installable to a home screen, no app store needed.
Audit log	Append-only record of who did what, when, with what result.

22 Sources

DATA CITED IN THIS DOCUMENT

1. Ministry of Textiles, Government of India — *Mapping of Textile Waste Value Chain in India*, released March 2026. India generates ~70.73 lakh tonnes of textile waste annually; 42% pre-consumer and 58% post-consumer; over 70% recovered overall, ~95% of pre-consumer and "nearly 55%" of post-consumer waste diverted from landfill.
2. Central Pollution Control Board — *Annual Report on Implementation of Solid Waste Management Rules, 2021–22*. India generates 1,70,339 TPD of municipal solid waste; 91,511 TPD processed, 41,455 TPD landfilled, 37,373 TPD unaccounted.

PROJECT MATERIALS

- **Master build prompt** — the instruction document for a build session (kept in the repo as `PROMPT.md`)
- `CLAUDE.md` — the live decision log; the authoritative record once the build starts
- **Earlier Django prototype** — `github.com/Fareeha-Amin/reloop-old-prototype`. Prior art and a source of the lessons in §19. Not a reference implementation.
- **IRJMETS paper** — early design snapshot, under review. Context only; see §20.

Keeping this document current

This is the stable layer: the problem, the mechanisms, the principles and the decisions. Day-to-day progress belongs in `CLAUDE.md`, not here. Revise this document when a *decision* changes — a new principle, a module promoted out of §17, a choice in §14 reversed — and bump the version on the cover.